

LLM-as-a-Judge Evaluation of Large Language Models on Ontology Design Pattern Generation: A Reproducible Multi-Dimensional Semiotic Assessment

ODPGen Research Team

April 24, 2026

1 LLM-as-a-Judge Evaluation of the Generated ODPs

This appendix reports a reproducible LLM-as-a-Judge evaluation of the Ontology Design Patterns generated in the main study. The evaluation pipeline source code lives in `eval_judge/` and its artefacts under `eval_judge_results/`.

1.1 Motivation

Traditional ontology-quality metrics rely on structural comparisons [6] or classification-based precision/recall against a reference graph [8]. These metrics penalise superficial lexical differences and cannot capture semantic equivalence under synonymy, nor functional adequacy against competency questions. The *LLM-as-a-Judge* paradigm [9] has been shown to correlate strongly with human judgement on open-ended generation tasks; the chain-of-thought variant G-Eval [10] further improves reliability by forcing the judge through explicit reasoning steps before emitting a score. We apply this paradigm to ODP evaluation, while addressing the two principal sources of bias identified in the literature: positional bias [11] and family-level self-enhancement bias [12].

1.2 Methodology

1.2.1 Dataset

We evaluate the 13 scenarios of the main study, each associated with a reference ground-truth ontology (GT) and a set of competency questions (CQs) [4]. GTs are serialised in Turtle, OWL/XML or RDF/XML; a format-agnostic loader (`eval_judge/data_loader.py`) handles all three. Five generative models and five prompt configurations are tested, yielding $5 \times 5 \times 13 = 325$ candidate ontologies after family-independence filtering (Section 1.2.4).

1.2.2 Evaluation Rubric

Drawing on the semiotic framework for ontology quality [6, 7], each candidate is scored on four dimensions plus an overall synthesis, each on a 1–10 integer scale:

Syntactic Correctness (structural). Valid OWL 2 DL/Turtle syntax, well-formed triples, declared prefixes, properly resolved IRIs.

Semantic Accuracy (functional). Concept precision and recall relative to GT, allowing for synonymy and naming variation.

Logical Consistency. Absence of unsatisfiable classes, contradictory axioms, or domain/range violations.

Functional Adequacy (pragmatics). Presence of the structural and axiomatic machinery required to answer each competency question.

Overall Score. Judge’s synthesis across the four dimensions.

The score bands are:

- 1–2 Syntactically invalid or logically incoherent; fails all CQs.
- 3–4 Major semantic drift; misses core classes; severe logical errors.
- 5–6 Basic taxonomy captured but complex restrictions missing.
- 7–8 Logically sound, highly similar semantics; answers almost all CQs.
- 9–10 Semantically isomorphic to GT; flawless logic; all CQs satisfied.

1.2.3 G-Eval Prompt

We adopt a chain-of-thought prompt following G-Eval [10]. The judge is instructed to (1) parse the GT to enumerate classes, properties and axioms; (2) assess syntactic validity; (3) compute semantic alignment allowing synonyms; (4) detect logical inconsistencies; (5) assess CQ answerability; and (6) synthesise an overall score. The final response must be a strict JSON object with one analysis string and one integer score per dimension. The full template is reproduced verbatim in Section 1.6.

1.2.4 Judge–Generator Family Independence

Self-enhancement bias [12] can inflate scores when the judge belongs to the same model family as the generator. We implement a conservative family check that matches by string substring (`gpt`, `claude`, `gemini`, `llama`, `mistral`, `bloom`). Evaluations where judge and generator share a family are skipped. In this study the judge is `gpt-4o` (OpenAI), so the 65 `gpt-5.4`-generated artefacts were excluded, yielding $N = 325$.

1.2.5 Bias Mitigation and Reproducibility

Determinism. Judge temperature is fixed to 0.0, `max_tokens` is 8192, and the JSON parser is robust to markdown fences and partial truncation (see `eval_judge/judge.py`).

Positional bias. Pairwise comparisons are run twice with the candidate positions swapped and their scores averaged [11]. Single-candidate evaluation — the primary mode used for Tables 1 to 4 — is immune to positional bias by construction.

Persistence. Every API response is saved verbatim alongside the parsed JSON under `eval_judge_results/{j}{config}/{scenario_id}.raw.txt`, enabling full re-analysis. Results are namespaced per judge model, allowing multiple judges to be compared without overwriting state.

1.2.6 Pipeline

The pipeline is implemented in Python and consists of three stages: (i) `run_evaluation.py` iterates over the `{model, config, scenario}` grid, calls the judge API, parses JSON, and caches per-instance results; (ii) `aggregate_results.py` produces per-model/config/scenario CSV tables and a Markdown report; (iii) `run_all_experiments.sh` orchestrates the full run, including dependency installation in an isolated `uv`-managed virtual environment. The one-line reproduction command is `./eval_judge/run_all_experiments.sh`, assuming a `.env` file with a valid `OPENAI_API_KEY`.

1.3 Results

1.3.1 Overall Scores by Model

Table 1 reports the mean ($\pm$ s.d.) of each dimension aggregated across the five prompt configurations and thirteen scenarios (65 observations per model).

Table 1: Judge scores by generating model (1–10 scale, $N = 65$ per row). Higher is better.

Model	Syntactic	Semantic	Logical	Functional	Overall
gemini-3.1-pro-preview	10.00 (0.00)	6.35 (1.40)	9.17 (0.52)	6.20 (1.80)	7.32 (1.26)
mistralai/Mistral-7B-Instruct-v0.3	9.95 (0.37)	5.43 (1.26)	8.75 (0.81)	5.22 (1.68)	6.49 (1.25)
meta-llama/Llama-3.1-8B-Instruct	9.85 (0.73)	5.34 (1.23)	8.62 (1.22)	5.06 (1.74)	6.34 (1.24)
meta-llama/Llama-2-70b-chat-hf	10.00 (0.00)	4.89 (1.30)	8.31 (0.98)	4.46 (1.61)	5.83 (1.14)
bigscience/bloomz-7b1	0.98 (0.12)	1.25 (0.79)	0.98 (0.12)	1.15 (0.59)	1.08 (0.44)

Gemini 3.1 Pro is the only model above 7 overall, reflecting high semantic recall together with near-perfect syntactic/logical form. The two smaller instruction-tuned open-source models (Mistral-7B-v0.3 and Llama-3.1-8B-Instruct) cluster tightly around 6.3–6.5 and outperform Llama-2-70B-Chat (5.83) by roughly half a point despite having an order of magnitude fewer parameters. BLOOMZ-7B essentially fails the task ($\bar{x} = 1.08$): inspection of raw outputs shows it frequently returns natural-language questions or input fragments rather than Turtle.

1.3.2 Effect of Prompt Configuration

Table 2: Judge scores by prompt configuration, aggregated across all five models ($N = 65$).

Config	Syntactic	Semantic	Logical	Functional	Overall
cq-only	8.20 (3.63)	4.97 (2.30)	7.20 (3.22)	4.82 (2.46)	5.74 (2.64)
scenario-cq	8.09 (3.63)	4.88 (2.16)	7.25 (3.26)	4.75 (2.28)	5.63 (2.53)
scenario-cq-reasoning	8.20 (3.63)	4.89 (2.20)	7.28 (3.26)	4.63 (2.37)	5.62 (2.56)
scenario-cq-constraints	8.20 (3.63)	4.80 (2.06)	7.09 (3.17)	4.83 (2.26)	5.60 (2.44)
scenario-only	8.09 (3.66)	3.72 (1.75)	7.02 (3.26)	3.06 (1.69)	4.48 (2.06)

Omitting CQs from the prompt (**scenario-only**) degrades overall performance by roughly 1.1 points, with the largest drops on *functional adequacy* (-1.77) and *semantic accuracy* (-1.25). The four CQ-inclusive configurations are statistically indistinguishable (within 0.14 points on overall), suggesting that additional reasoning or constraint cues on top of the basic CQ list provide no detectable benefit at these model scales.

1.3.3 Cross-Tabulation: Model $\times$ Configuration

Table 3: Mean overall score per (model, prompt configuration) cell, $n = 13$ per cell.

Model	cq-only	scenario-cq	scen-cq-constr.	scen-cq-reas.	scen-only
bloomz-7b1	1.1	1.0	1.3	1.1	0.9
gemini-3.1-pro	7.7	7.5	7.5	7.6	6.3
Llama-2-70B-Chat	6.2	6.0	6.0	5.8	5.2
Llama-3.1-8B-Inst	6.9	6.7	6.5	6.7	4.8
Mistral-7B-v0.3	6.8	6.9	6.7	6.8	5.2

The **scenario-only** degradation is non-uniform across models. Llama-3.1-8B suffers a steep -2.1 drop (from 6.9 to 4.8), while Llama-2-70B only loses 1.0. This is consistent with differing

reliance on explicit task specification across instruction-tuning recipes: newer recipes appear to exploit CQ information more aggressively.

1.3.4 Variability by Scenario

Table 4: Per-scenario score means ($N = 25$: 5 models $\times$ 5 configurations). Std deviations omitted for brevity.

Scenario	Syntactic	Semantic	Logical	Functional	Overall
2023-133-01	8.08	4.28	6.68	4.16	5.16
2023-133-02	8.20	4.20	7.08	3.92	5.04
2023-134-01	8.20	4.52	7.40	4.08	5.12
2023-134-02	8.16	5.08	7.72	5.04	5.80
2023-134-03	8.20	4.32	7.48	4.04	5.08
2023-135-01	8.20	3.92	7.24	4.36	5.16
2025-147-01	8.20	4.04	7.20	3.40	4.84
2025-149-01	8.20	4.92	7.36	4.36	5.60
2025-150-01	8.20	4.20	7.04	3.64	5.04
2025-151-01	7.96	6.24	7.56	6.40	6.88
2025-151-02	8.20	5.28	7.20	5.04	5.80
2025-153-01	8.04	4.36	6.16	4.12	5.08
2026-155-01	8.20	5.12	7.04	4.88	5.76

The easiest scenario is 2025-151-01 (evaluation-request pattern, overall 6.88); the hardest is 2025-147-01 (context-dependent signal pattern, 4.84). Variability across scenarios ($\sigma = 0.54$ in overall means) is notably smaller than variability across models, indicating that the primary determinant of success is the generator rather than the target scenario.

1.4 Discussion

1.4.1 A Syntax–Semantics Gap

The most striking pattern is the persistent gap between syntactic correctness (mean ≈ 10 for all competent models) and semantic accuracy (mean ≈ 5). Modern instruction-tuned LLMs have essentially solved the surface form of Turtle but still struggle to model domains faithfully. This mirrors findings in code generation: compilable code is common, but semantic equivalence to a specification remains elusive.

1.4.2 Smaller Instruction-Tuned Models Outperform Larger Base-Era Models

Llama-3.1-8B-Instruct and Mistral-7B-v0.3 (≈ 6.4 overall) beat Llama-2-70B-Chat (5.83) despite an $\approx 9\times$ parameter disadvantage. This is consistent with the hypothesis that instruction-tuning recipes and data quality matter more than raw parameter count for formal/structural tasks, and reaffirms the rapid progress between Llama-2 and Llama-3 [18].

1.4.3 Prompt Configuration Effects Are Modest When CQs Are Present

Once CQs are included, adding reasoning instructions or explicit constraint cues yields no measurable improvement. This could reflect either (a) a ceiling imposed by the generator’s formal ontology-engineering capability rather than prompt-following ability, or (b) a need for richer chain-of-thought prompting than the simple cue strings used here. **scenario-only** mode, however, strongly underperforms, underscoring the centrality of CQs as specification.

1.4.4 BLOOMZ-7B Failure Mode

BLOOMZ’s overall score (1.08) is not a scoring artefact: its raw responses frequently contain natural-language questions or fragments of the input prompt rather than Turtle. Since BLOOMZ was not instruction-tuned on structured-output tasks, this reflects a data-distributional rather than capacity limitation.

1.4.5 Threats to Validity

Judge bias. Although we enforce judge-generator family independence, GPT-4o may share latent stylistic preferences with non-family peers. The gap between GPT-4o’s scores and those of a prospective Claude or Gemini judge will be measured by re-running the pipeline with `claude-sonnet-4-6` and `gemini-2.5-pro` judges; the per-judge directory layout supports this natively.

Single-candidate mode. The scores reported here come from single-candidate evaluation. Positional bias is absent by construction, but absolute-score calibration may still be judge-specific [10].

Coverage. Only 13 scenarios are evaluated here. Expanding to the full ODP-portal catalogue is a clear next step and is feasible given the pipeline’s $\mathcal{O}(n)$ API cost.

Family-independence exclusion. Because the judge is GPT-4o, the 65 `gpt-5.4` candidate evaluations were excluded. A cross-judge ensemble would permit their inclusion without introducing self-enhancement bias.

1.5 Summary

A focused study of five generators, five prompt configurations, and thirteen scenarios ($N = 325$) reveals a clear capability ordering dominated by Gemini 3.1 Pro, with modern 7–8 B instruction-tuned open-source models approaching but not matching proprietary performance. Including competency questions in the prompt is clearly beneficial; further prompt elaboration (reasoning/constraint cues) offers no measurable gain at these model scales. All artefacts — raw judge responses, per-instance JSON, and aggregated CSV/Markdown reports — are preserved in `eval_judge_results/gpt-4o/` for reproducibility.

1.6 Evaluation Prompts

This section reproduces verbatim the prompts issued to the judge LLM. At inference time the placeholders `{ground_truth}`, `{competency_questions}`, and `{candidate}` (resp. `{candidate_a}`, `{candidate_b}`) are substituted with the scenario-specific content. The authoritative source lives in `eval_judge/prompt_templates.py`.

1.6.1 System Prompt

The system prompt is shared by both single-candidate and pairwise modes:

```
1 You are an expert Semantic Web Logician and Ontology
2 Engineer. You evaluate candidate Ontology Design Patterns
3 (ODPs) against a Ground Truth ODP using a rigorous,
4 multi-dimensional rubric.
```

1.6.2 Single-Candidate Evaluation Prompt

This prompt produced all $N = 325$ scores in Section 1.3. It implements the G-Eval Chain-of-Thought paradigm [10] with six ordered reasoning steps.

```
1 Evaluate the following Candidate Ontology Design Pattern against
2 the Ground Truth ODP on four dimensions and assign a 1-10 score
3 for each dimension plus an overall score.
4
5 === INPUTS ===
6
7 [Ground Truth ODP]:
8 {ground_truth}
9
10 [Competency Questions]:
11 {competency_questions}
12
13 [Candidate ODP]:
14 {candidate}
15
16 === EVALUATION STEPS (follow in order) ===
17
18 1. **Parse the Ground Truth**: Identify core classes, object/data
19    properties, axioms (domain/range, cardinality, restrictions),
20    and the overall modelling intent.
21
22 2. **Syntactic Correctness** (Structural): Verify the Candidate
23    uses valid OWL 2 DL / RDF Turtle syntax. Check for malformed
24    triples, undeclared prefixes, or broken URIs.
25
26 3. **Semantic Accuracy** (Functional): Compare the Candidate's
27    classes and properties against the Ground Truth. Compute
28    conceptual precision (how many Candidate concepts are relevant)
29    and recall (how many Ground Truth concepts are captured).
30    Allow valid synonyms and minor naming differences.
31
32 4. **Logical Consistency**: Detect unsatisfiable classes,
33    contradictory axioms, domain/range violations, or circular
34    definitions in the Candidate.
35
36 5. **Functional Adequacy** (Pragmatics): For each Competency
37    Question, determine whether the Candidate contains the
38    logical structure (classes, properties, axioms) required
39    to answer it.
40
41 6. **Overall Score**: Synthesise dimensions 2-5 into an overall
42    quality score.
43
44 RUBRIC (1-10 scale):
45   1-2 : Syntactically invalid or logically incoherent.
46         Fails all CQs.
47   3-4 : Major semantic drift, misses core classes,
48         severe logical errors.
49   5-6 : Captures basic taxonomy but misses complex
50         restrictions. Fails complex CQs.
51   7-8 : Logically sound, highly similar semantics.
52         Answers almost all CQs. Minor structural divergences.
53   9-10: Semantically isomorphic to the Ground Truth.
54         Flawless logic, satisfies all CQs.
55
56 === OUTPUT FORMAT ===
57 Respond strictly as a JSON object - no markdown fences,
58 no extra text:
59 {
```

```

60 "syntactic_correctness_analysis": "<reasoning>",
61 "syntactic_correctness_score": <int 1-10>,
62 "semantic_accuracy_analysis": "<reasoning>",
63 "semantic_accuracy_score": <int 1-10>,
64 "logical_consistency_analysis": "<reasoning>",
65 "logical_consistency_score": <int 1-10>,
66 "functional_adequacy_analysis": "<reasoning>",
67 "functional_adequacy_score": <int 1-10>,
68 "overall_analysis": "<1-2 sentence summary>",
69 "overall_score": <int 1-10>
70 }

```

1.6.3 Pairwise Evaluation Prompt (with Position-Swap Debiasing)

The pairwise variant is used only in the pairwise runs kept under `eval_judge_results/gpt-4o/pairwise/`. Each pair is scored twice with Candidate A and Candidate B swapped, and the two scores are averaged to cancel positional bias [11].

```

1 Evaluate two candidate Ontology Design Patterns (Candidate A and
2 Candidate B) against a Ground Truth ODP on a scale of 1 to 10.
3
4 === INPUTS ===
5
6 [Ground Truth ODP]:
7 {ground_truth}
8
9 [Competency Questions]:
10 {competency_questions}
11
12 [Candidate A]:
13 {candidate_a}
14
15 [Candidate B]:
16 {candidate_b}
17
18 === EVALUATION STEPS (follow in order) ===
19
20 1. **Parse the Ground Truth** to identify core classes,
21    properties, and axioms.
22
23 2. **Syntactic Correctness**: Verify valid OWL/RDF syntax
24    for both candidates.
25
26 3. **Semantic Alignment**: Compare Candidate A and B
27    concepts against the Ground Truth (allow valid synonyms).
28
29 4. **Logical Consistency**: Identify any unsatisfiable
30    classes or contradictory axioms.
31
32 5. **Functional Adequacy**: Assess if the candidates contain
33    the structure to answer the Competency Questions.
34
35 6. **Scoring**: Assign a 1-10 score for each candidate based
36    on the rubric.
37
38 RUBRIC (1-10 scale):
39 1-2 : Syntactically invalid or logically incoherent.
40      Fails all CQs.
41 3-4 : Major semantic drift, misses core classes,
42      severe logical errors.
43 5-6 : Captures basic taxonomy but misses complex
44      restrictions. Fails complex CQs.

```

```

45 7-8 : Logically sound, highly similar semantics.
46     Answers almost all CQs. Minor structural divergences.
47 9-10: Semantically isomorphic to the Ground Truth.
48     Flawless logic, satisfies all CQs.
49
50 === OUTPUT FORMAT ===
51 Respond strictly as a JSON object - no markdown fences,
52 no extra text:
53 {
54   "Candidate_A_Analysis": "<step-by-step reasoning for A>",
55   "Candidate_A_Score": <int 1-10>,
56   "Candidate_B_Analysis": "<step-by-step reasoning for B>",
57   "Candidate_B_Score": <int 1-10>
58 }

```

1.6.4 API Call Parameters

All judge API calls use the following parameters (defined in `eval_judge/judge.py`):

Parameter	Value
Judge model	<code>gpt-4o</code>
Temperature	<code>0.0</code>
<code>max_tokens</code>	<code>8192</code>
Retries on transient errors	<code>3</code> (5 s back-off)

References

- [1] A. Gangemi and V. Presutti. Ontology Design Patterns. In *Handbook on Ontologies*, 2nd ed., pp. 221–243, Springer, 2009.
- [2] V. Presutti and A. Gangemi. Content Ontology Design Patterns as Practical Building Blocks for Web Ontologies. In *ER 2008*, LNCS 5231, pp. 128–141, Springer, 2008.
- [3] E. Blomqvist, K. Hammar, and V. Presutti. Engineering Ontologies with Patterns: The eXtreme Design Methodology. In *Ontology Engineering with Ontology Design Patterns*, Studies on the Semantic Web 25, IOS Press, 2016.
- [4] M. Grüninger and M. S. Fox. Methodology for the Design and Evaluation of Ontologies. In *IJCAI’95 Workshop on Basic Ontological Issues in Knowledge Sharing*, 1995.
- [5] W3C OWL Working Group. OWL 2 Web Ontology Language Primer (Second Edition). W3C Recommendation, 2012. <https://www.w3.org/TR/owl2-primer/>
- [6] A. Gangemi, C. Catenacci, M. Ciaramita, and J. Lehmann. Modelling Ontology Evaluation and Validation. In *ESWC 2006*, LNCS 4011, pp. 140–154, Springer, 2006.
- [7] C. Burton-Jones, V. C. Storey, V. Sugumaran, and P. Ahluwalia. A Semiotic Metrics Suite for Assessing the Quality of Ontologies. *Data & Knowledge Engineering*, 55(1):84–102, 2005.
- [8] D. Dell’Aglia, D. Calvanese, and J. Hendler. Evaluating the Quality of Ontologies: A Survey. *Semantic Web*, 11(1):1–25, 2020.
- [9] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS*, 2023.

- [10] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu. G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment. *EMNLP*, 2023.
- [11] P. Wang, L. Li, L. Chen, Z. Cai, D. Zhu, B. Lin, Y. Cao, Q. Liu, T. Liu, and Z. Sui. Large Language Models are not Fair Evaluators. *arXiv:2305.17926*, 2023.
- [12] A. Panickssery, S. Bowman, and S. Feng. LLM Evaluators Recognize and Favor Their Own Generations. *NeurIPS*, 2024.
- [13] M. Saeedizade and E. Blomqvist. NeOn-GPT: A Large Language Model-powered Pipeline for Ontology Learning. *ESWC Demo Track*, 2024.
- [14] N. Fathallah, A. Das, S. D. Giorgis, A. Poltronieri, P. Haase, and L. Kovriguina. NeOn-GPT: A Large Language Model-powered Pipeline for Ontology Learning. *arXiv:2410.05828*, 2024.
- [15] N. Mihindukulasooriya, S. Tiwari, C. F. Enguix, and K. Lata. Text2KGBench: A Benchmark for Ontology-Driven Knowledge Graph Generation from Text. *ISWC*, 2023.
- [16] H. Babaei Giglou, J. D’Souza, and S. Auer. LLMs4OL: Large Language Models for Ontology Learning. *ISWC*, 2023.
- [17] S. Capadisli, T. Berners-Lee, R. Verborgh, and K. Kjernsmo. Solid Protocol. W3C Editor’s Draft, 2020. <https://solidproject.org/TR/protocol>
- [18] A. Dubey et al. The Llama 3 Herd of Models. *arXiv:2407.21783*, 2024.